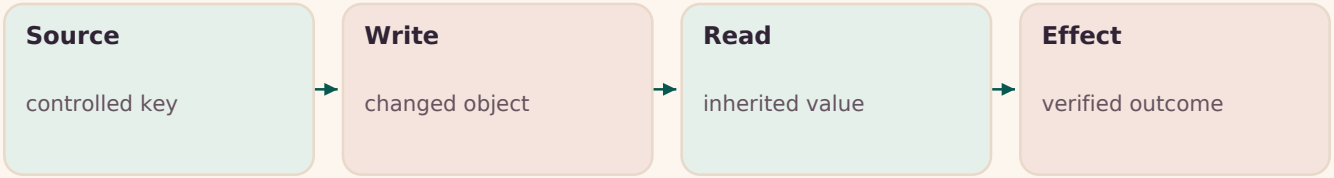
PROTOTYPE POLLUTION EXPLAINED

From JavaScript objects to finding, proving, and preventing inheritance attacks

THE FIELD MODEL



Follow the key. Find the shared object. Prove what changed.

Haroon Shahid

Introduction, roadmap and Chapters 1-3

NAVIGATION

Contents

Choose a chapter. The footer on reading pages returns here.

01 Values, references and identity

02 Objects, keys and methods

03 Functions, scope and time

Reading roadmap • Next steps

ORIENTATION

Read the object graph

Learn the language rules first. Then follow a controlled key until it produces a verified effect.

Lantern Desk is a fictional document workspace. Its editors can choose display preferences. They should not get to edit the shared defaults or choose the facts that grant permission. We will follow one input from text, through an object graph, to the exact decision that changes.

CENTRAL QUESTION

Who controlled this key, what object changed, who can read it, and why did the program act on that value?

1. Objects and data: Draw separate object identities and predict when a write is shared.

2. Inheritance and the runtime: Find the first descriptor, explain the receiver, and predict the exact read or write.

3. Where unsafe writes start: Identify the first input-driven branch selection and classify which object changed.

4. Discovery and impact: Demonstrate the source and consumer independently and write a bounded impact statement.

5. Prevention and repair: Repair the source and reader, then show valid inputs still work.

6. Advanced review and capstone: Reproduce, repair and report the capstone with enough evidence for another reader to repeat it.

Predict each example before running it. Compare clean, weak and repaired observations. Keep source, write, scope, read and effect separate in your notes. If you are new to JavaScript, begin at Chapter 1 and postpone the lab until you can draw the first object graph.

This sample contains the opening three chapters. The complete edition adds discovery, browser and Node examples, secure redesign, the workbook and the local lab.

Edition 1.0. Prepared with AI assistance. Contact: gaknt2@gmail.com.

Values, references and identity

Predict when changing one variable's object changes what another variable sees.

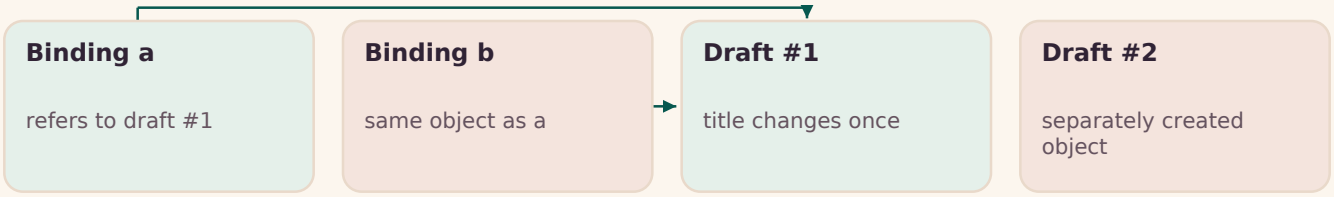
START HERE

Lantern Desk is a small app that turns notes into documents. Mira opens the same draft in two panels. A title change in one panel appears in the other. Before calling it a bug, ask a better question: do both panels refer to the same draft?

A value is something JavaScript can use, such as the number 3, the string "draft", or an object. A variable is a named binding to a value. When you assign one object-valued variable to another, both bindings hold the same object value. The program does not make a second object. Changing a property through one name can affect every other place that refers to that object.

Two objects with equal-looking properties can still be different objects. With objects, === compares identity. With the strings and numbers used here, it compares values of the same type. JavaScript passes values into functions; passing an object value gives the function access to that same object. Reassigning a parameter does not reassign the caller's variable.

RUNTIME MAP



Before the write, a and b refer to object #1. c refers to object #2. JavaScript writes title on #1. Reading b.title reaches #1. Reading c.title reaches #2. The diagram describes language behavior; engines need not store these values at particular physical addresses.

A PICTURE TO REMEMBER

Two claim tickets can point to one coat. Altering the coat changes what either ticket retrieves. Replacing one ticket with a ticket for a different coat leaves the other ticket alone.

Same object, different name

```
const a = { title: "Draft" };
const b = a;
const c = { title: "Draft" };
b.title = "Ready";
console.log(a.title, c.title); // Ready Draft
console.log(a === b, a === c);  // true false
```

WHAT COUNTS AS EVIDENCE

b.title changing does not prove a change in shared inheritance. First determine whether a and b are the same object. Later you will see two genuinely different objects share a separate prototype.

COMMON MISTAKE

"Objects are passed by reference" can imply a function can rebind the caller's variable. A function can mutate the object it receives, but local rebinding and mutation are different operations.

TRY IT

Predict the two titles if the write is c.title = "Ready". Draw a second object and put each arrow on the correct identity before running the example.

KNOWLEDGE CHECK / ANSWER BEFORE READING ON

1. Which identity does b refer to after const b = a? The same object as a. Assignment does not clone an object.

2. Why does c keep its first title? It refers to a separately created object.

3. A function assigns its parameter to {}. Did it replace the caller's object? No. It changed only its local binding.

TAKEAWAY

Check identity first. Shared state can come from aliases or from a shared prototype; those are different routes.

Sources: S01 · S02

Objects, keys and methods

Read object syntax and separate keys, values, and operations.

START HERE

Lantern Desk stores a title and a display method. A user may choose the title. The app chooses the method. Both are properties, but the app should grant control over only the first one.

An object can have properties. Each property has a key and a value or accessor behavior. A key is a string or a symbol. Numeric bracket keys such as 7 normally become strings like "7". Dot syntax uses a literal name; brackets compute the key from an expression. A method is a function used as a property; calling it with obj.method() normally supplies obj as this for an ordinary function.

Arrays are objects with special behavior for index keys and length. They still have properties and inherit methods. Do not assume that data sitting in an array escaped security rules for objects. If your input contract says "list of strings," check that shape; accepting any object with numeric-looking keys changes the contract.

RUNTIME MAP

Object	Key "title"	Value "Draft"	Method
record with properties	selects a property	app data	function stored as a property

const key = "title" gives the bracket lookup a string. draft[key] uses that string. draft.key instead asks for the literal name "key". A computed key becomes a trust boundary when it came from a request and the code uses it to select a write target.

A PICTURE TO REMEMBER

A document cabinet has labelled drawers. The label and the contents are separate. Letting a visitor write a note for one drawer is different from letting them choose any label in the cabinet.

Dot and bracket access

```
const draft = {
  title: "Draft",
  label() { return this.title; }
};
const field = "title";
console.log(draft[field], draft.label());
console.log(draft.field); // undefined
```

WHAT COUNTS AS EVIDENCE

A dynamic key is not automatically unsafe. A fixed allowlist mapping "title" to an approved write is different from accepting every key. Ask which keys the input can choose and which object receives them.

COMMON MISTAKE

Values and keys have different security jobs. Restricting title length cannot prevent a user from choosing a dangerous key elsewhere.

TRY IT

Predict draft["label"] and draft["label"](). Say which expression retrieves a function and which calls it.

KNOWLEDGE CHECK / ANSWER BEFORE READING ON

1. What does draft[field] read? The name stored in field; here it is title.

2. Is an array unrelated to object inheritance? No. Arrays are objects with array behavior and a prototype chain.

3. Is every method defined directly on the object using it? No. An object can call an inherited function as a method.

TAKEAWAY

Treat untrusted keys as instructions about where data goes. Validate them separately from the data itself.

Sources: S01 · S02

Functions, scope and time

Follow data across a function call and tell local state from shared state.

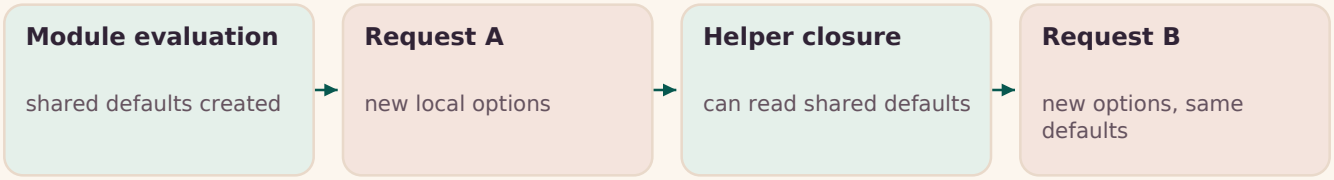
START HERE

Lantern Desk assigns each export a new options object, but loads a single defaults object once. A helper writes to both during development. Which state survives the next export?

A function groups instructions. A parameter names an argument received on a call. Scope says where names can be used. Variables declared inside a function generally have fresh bindings on each call. A variable outside a function can be read by several calls. A closure is a function that retains access to bindings from its surrounding code.

This matters for security because "new request" does not mean "new JavaScript process." A server can reuse loaded modules, objects, and built-in prototypes for many requests. An async function can pause and resume later; other work can occur in between. JavaScript executing one turn at a time does not make shared mutations safe across turns.

RUNTIME MAP



At startup one shared object exists. Each render creates another local object. The helper mutates the shared object. The next call creates a new local object, then still reads the modified shared object. Track lifetime as well as identity.

A PICTURE TO REMEMBER

Each guest gets a fresh order slip. The restaurant also has one shared menu board. Editing a slip affects one order; editing the board changes what later guests see.

A fresh call and an old object

```
const defaults = { paper: "A4" };
function render() {
  const job = { copyCount: 1 };
  return [job.copyCount, defaults.paper];
}
console.log(render()); // [1, "A4"]
defaults.paper = "Letter";
console.log(render()); // [1, "Letter"]
```

WHAT COUNTS AS EVIDENCE

If a second request changes after the first, a persistent object is one explanation. Other candidates include sessions, caches, DB rows, and load-balancer routing. A reliable test needs controls to separate them.

COMMON MISTAKE

const stops rebinding, not property writes. const defaults is still mutable unless the object itself is constrained.

TRY IT

Move defaults inside render and predict both results after removing the outside write. Then consider what happens if a returned job is stored in a module-level array.

KNOWLEDGE CHECK / ANSWER BEFORE READING ON

1. Does const freeze an object? No. It prevents assigning a new value to that binding.

2. Can a closure read a binding after the outer function returns? Yes, if the closure retains access to it.

3. Does one event loop make cross-request mutations impossible? No. Requests can run at different times and see the same shared state.

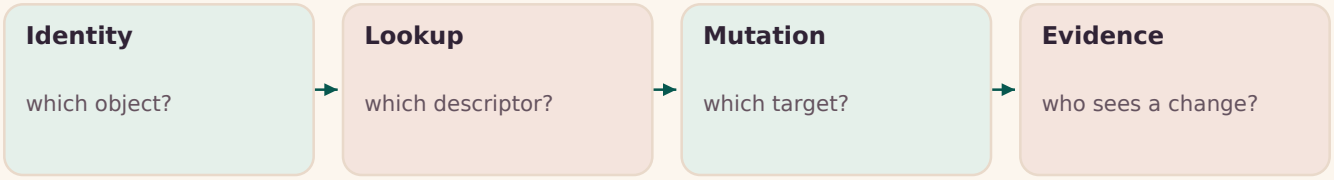
TAKEAWAY

Draw two timelines: when an object is created, and when each request or function can still reach it.

Sources: S02 · S13

Continue the investigation

You can now distinguish two variables referring to the same object from two objects that happen to contain the same values. Continue by predicting what a prototype read will do when the receiver has no own descriptor.



The full learning kit adds all 36 chapters, 108 answered checks, a workbook, revision atlas, case library and isolated local lab. Look for the full edition through the catalog where you found this preview.

This preview may be shared as an unmodified sample with attribution. For help or corrections, email gaknt2@gmail.com.